

CloudShield AI / DAIL

07 — Identity & Dependency Specification

Version 1.0 | Final Identity/Dependency Algorithm Baseline | 1 September 2026

Purpose: Define the deterministic algorithms, matching rules, dependency semantics, ambiguity handling, graph construction rules, and evidence requirements used by DAIL to determine what changed and what is connected within the supported Terraform/AWS model.

Document Control

Field	Value
Document ID	CSAI-DAIL-IDDEP-007
Status	FINAL ALGORITHM BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle
Primary outputs	IdentityReport and DependencyGraph
Initial algorithm	Deterministic rule-based matching and graph extraction
LLM authority	None for identity/dependency facts
Core principle	Missing knowledge is never treated as proof of absence

1. Scope

This specification owns resource correspondence and supported dependency graph construction. It does not decide invariant impact, verification, or promotion.

- Match Trusted-State resources to Candidate-State resources.
- Classify unchanged, modified, created, deleted, replaced, and uncertain changes.
- Extract Terraform references.
- Construct supported infrastructure relationships.
- Construct invariant scope/semantic relationships.
- Preserve ambiguity and uncertainty.
- Produce deterministic, explainable reports.

2. Identity Model

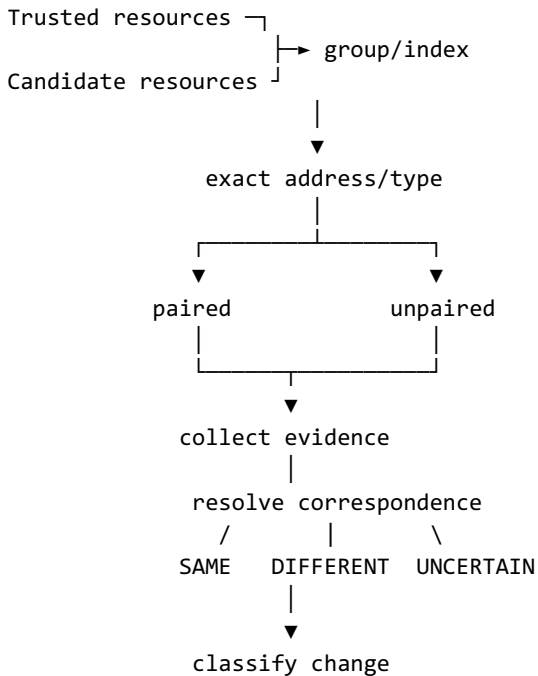
Concept	Definition
Source identity	Terraform address in configuration/module context.
Semantic identity	Identity established from resource type, scope, controlled attributes, and supported provider semantics.
Provider identity	AWS/provider-assigned identifier where reliable.
Fingerprint	Hash of canonical semantic representation.
Correspondence	Supported claim that candidate represents the same logical resource as trusted resource.
Replacement	Existing resource is recreated/replaced rather than merely modified.
Uncertainty	Evidence is insufficient or conflicting for safe

	correspondence.
--	-----------------

3. Identity Inputs

- Normalized Trusted-State resources.
- Normalized Candidate-State resources.
- Canonical Terraform addresses and types.
- Canonical fingerprints.
- Logical identity fields.
- Provider/runtime IDs where available.
- Terraform plan actions where available.
- Provenance and normalization version.

4. Matching Pipeline



5. Matching Priority

Priority	Evidence	Interpretation
1	Exact canonical address + same supported type	Strong correspondence starting point.
2	Provider/runtime ID + compatible type/context	Strong correspondence when stable.
3	Stable logical identity + scope/type	Supporting correspondence evidence.
4	Terraform plan replacement/action metadata	Strong change evidence when available.
5	Fingerprint/canonical semantic comparison	Detects semantic change; does not independently prove identity.
6	Heuristic similarity	Candidate discovery only; never

		sufficient alone for SAME.
--	--	----------------------------

A weaker heuristic may not override contradictory stronger evidence.

6. Exact Address Rule

```

if trusted.address == candidate.address
and trusted.type == candidate.type:
    establish provisional pair
    inspect replacement/change evidence

```

Same address is not an unconditional guarantee of in-place identity. Explicit replacement evidence may classify the transition as REPLACED.

7. Provider Identity Rule

- Equal provider ID + compatible resource context is strong SAME evidence.
- Provider ID change may indicate replacement when the provider semantics support that conclusion.
- Provider IDs unavailable during configuration-only analysis are simply unavailable, not negative evidence.
- Provider IDs must be interpreted within account/region/resource-type context.

8. Logical Identity Context

Resource	Relevant identity evidence
VPC	Address/type + controlled provider identity/scope.
Subnet	Address/type + VPC context + provider identity where available.
Route table	Address/type + VPC context + provider identity.
EC2	Address/type + provider identity/context where available.
Security group	Address/type + VPC scope + provider identity.
Security group rule	Owning SG + canonical rule semantics + supported identity.
RDS	Address/type + provider identity + placement/context.

9. Fingerprint Rules

Relation	Result
Equal fingerprint + established correspondence	UNCHANGED.
Different fingerprint + established correspondence	MODIFIED unless replacement evidence exists.
Candidate has no trusted correspondence	CREATED.
Trusted has no candidate correspondence	DELETED.
Explicit recreation/replacement evidence	REPLACED.
Conflicting/insufficient identity evidence	UNCERTAIN.

10. Identity Decision Matrix

Trusted	Candidate	Evidence	Result
Yes	Yes	Same address/type; equal semantics	SAME + UNCHANGED
Yes	Yes	Same address/type;	SAME + MODIFIED

		semantic difference	
Yes	Yes	Replacement evidence	SAME correspondence + REPLACED
No	Yes	No trusted match	CREATED
Yes	No	No candidate match	DELETED
Yes	Yes	Conflicting evidence	UNCERTAIN
Yes	Yes	Only weak heuristic evidence	UNCERTAIN
Yes	Yes	Unsupported semantics	UNSUPPORTED/UNCERTAIN; never optimistic

11. Ambiguity Handling

If multiple candidates plausibly correspond to one trusted resource, or vice versa, the engine must not select arbitrarily.

```

1 trusted → 2+ plausible candidates
    => ambiguous correspondence
    => UNCERTAIN
    => preserve all supporting evidence

```

Ambiguity is a first-class output and must be visible to downstream impact/promotion logic.

12. Replacement Detection

- Use explicit Terraform plan replacement actions where available.
- Use stable address + provider-ID change only when provider semantics support it.
- Use documented resource-specific replace-trigger fields only when included in the controlled model.
- If evidence is insufficient, retain CREATED/DELETED or UNCERTAIN rather than inventing REPLACED.

13. Identity Report Contract

```

IdentityReport {
  parent_state_id
  candidate_id
  resource_matches[]
  summary {
    unchanged[]
    modified[]
    created[]
    deleted[]
    replaced[]
    uncertain[]
  }
  algorithm_version
  evidence[]
}

```

14. Dependency Model

Edge class	Meaning
Terraform reference	Source attribute explicitly references target.
Infrastructure relationship	Supported AWS/Terraform relationship derived from semantics.
Invariant scope	Invariant applies to a resource/property set.
Invariant semantic relationship	Invariant connects multiple resources/semantics, e.g. EC2–RDS.

15. Terraform Reference Extraction

```
for each normalized resource attribute:
  find supported resource references
  canonicalize target address
  resolve target
  create Reference
  emit dependency edge
```

- Reference provenance includes source resource and attribute.
- Unresolved references become UNKNOWN/UNSUPPORTED according to model.
- References are not silently discarded.

16. Supported Infrastructure Edges

Source	Target	Edge type
aws_subnet	aws_vpc	SUBNET_BELONGS_TO_VPC
aws_route_table	aws_vpc	ROUTE_TABLE_BELONGS_TO_VPC
aws_route_table_association	aws_subnet	ASSOCIATES_SUBNET
aws_route_table_association	aws_route_table	ASSOCIATES_ROUTE_TABLE
aws_internet_gateway	aws_vpc	IGW_ATTACHED_TO_VPC
aws_instance	aws_subnet	INSTANCE_IN_SUBNET
aws_instance	aws_security_group	INSTANCE_USES_SECURITY_GROUP
aws_db_instance	aws_security_group	DB_USES_SECURITY_GROUP
aws_security_group_rule	aws_security_group	RULE_BELONGS_TO_SECURITY_GROUP

17. Routing Relationships

- Subnet-to-route-table association identifies the effective route-table context.
- Route destination/target relationships are represented only within the supported routing model.
- Internet-gateway routes create supported internet-path relationships where configured.
- NAT semantics are not inferred unless explicitly supported.
- Unknown route targets remain unknown/unsupported.

A routing edge is configuration/semantic evidence; it is not automatically proof of successful packet delivery.

18. Security-Group Relationships

```
EC2 —USES_SG→ AppSG
RDS —USES_SG→ DbSG
```

Rule —BELONGS_TO→ SG

Rule nodes/edges retain canonical rule semantics so downstream analysis can identify which security obligations may be affected.

19. Invariant Relationships

Invariant	Graph representation
INV-SEC-001	Invariant node connected to scoped security-group/rule/resource nodes.
INV-FUNC-001	Invariant node connected to EC2 source, RDS destination, TCP/5432 semantics, and relevant supported network/security context.

The invariant registry owns the definition and scope; this document defines how declared scope is represented.

20. Graph Construction Contract

```
build_graph(state, invariant_registry):  
    add resource nodes using stable semantic IDs  
    extract explicit references  
    derive supported infrastructure edges  
    derive invariant scope/semantic edges  
    mark uncertain/unsupported relationships  
    sort nodes/edges deterministically  
    compute canonical graph representation/hash  
    return DependencyGraph
```

21. Graph Node Identity

Resource node:
RESOURCE:<canonical Terraform address>

Invariant node:
INVARIANT:<id>:<version>

Database UUIDs are persistence identifiers and are not semantic graph IDs.

22. Graph Edge Identity

```
edge_key =  
    SHA256(source_node + edge_type + target_node + canonical_edge_attributes)
```

This makes graph comparison and evidence references deterministic.

23. Dependency Uncertainty

Condition	Required behavior
Reference cannot resolve	UNKNOWN/UNRESOLVED edge; retain provenance.

Relationship not supported	UNSUPPORTED; never infer no relationship.
Conflicting relationship facts	UNCERTAIN.
Optional metadata absent but relationship established	Edge may remain KNOWN.
Provider behavior outside model	UNCERTAIN/UNSUPPORTED.

24. No-Edge Rule

NO EDGE $\neq$ NO DEPENDENCY
 NO EVIDENCE $\neq$ EVIDENCE OF NO DEPENDENCY

The graph represents supported knowledge, not a complete proof of infrastructure independence.

25. Dependency Closure Boundary

This engine constructs the graph only. It does not decide traversal depth or invariant invalidation. Document 08 consumes this graph to compute affected obligations.

- Direct references are represented.
- Supported semantic relationships are represented.
- Uncertainty is retained.
- Graph is available for impact analysis.

26. Identity Evidence

```
IdentityEvidence {
  trusted_ref
  candidate_ref
  evidence_type
  source
  observed_value
  interpretation
  algorithm_version
}
```

Each identity result should be explainable as evidence rather than an opaque score.

27. Dependency Evidence

```
DependencyEvidence {
  source
  target
  edge_type
  source_fact
  target_fact
  derivation_rule
  provenance
  status
}
```

28. Determinism Requirements

- Sort resources by canonical address.
- Use deterministic indexes and tie handling.
- Sort graph nodes/edges by stable semantic IDs.
- Use deterministic report serialization.
- Never let iteration order resolve an ambiguity.
- Never use LLM output as a deterministic identity/dependency decision.

29. Algorithm Versioning

Component	Requirement
Identity engine	Identity algorithm version in every report.
Dependency extractor	Extractor/model version in every graph.
Normalization	Must match Document 04 normalization version.
Invariant registry	Invariant definition/version recorded in invariant graph nodes.

30. Identity Pseudocode

```
def compare(trusted, candidate, plan=None):
    T = index(trusted)
    C = index(candidate)
    results = []

    for key in sorted(union(T.keys(), C.keys())):
        t, c = T.get(key), C.get(key)

        if t and c:
            evidence = collect_identity_evidence(t, c, plan)
            status = resolve_identity(evidence)
            change = classify_change(t, c, evidence, status)
            results.append(match(t, c, status, change, evidence))
        elif c:
            results.append(created(c))
        else:
            results.append(deleted(t))

    resolve_cross_address_ambiguity(results)
    return build_report(results)
```

31. Dependency Pseudocode

```
def build_graph(state, invariants):
    graph = Graph()

    for resource in sorted(state.resources, key=canonical_address):
        graph.add_resource_node(resource)
        for ref in resource.references:
            graph.add_reference_edge(ref)
        for edge in derive_supported_relationships(resource, state):
            graph.add_edge(edge)
```



```

for invariant in invariants:
    graph.add_invariant_node(invariant)
    for edge in derive_scope_edges(invariant, state):
        graph.add_edge(edge)

graph.normalize()
return graph

```

32. Identity vs Dependency vs Impact

Identity:
 "aws_security_group_rule.app_ssh_public → MODIFIED"

Dependency:
 "rule → app security group → EC2"

Impact:
 "INV-SEC-001 → AFFECTED"

Only the first two statements are owned by this specification.

33. Canonical Scenario

Trusted:
 Rule app_ssh_public → belongs to AppSG → used by EC2

Candidate:
 same address, changed canonical rule

Identity:
 SAME + MODIFIED

Dependency:
 Rule → SG → EC2
 Rule → INV-SEC-001

The resulting reports give Document 08 enough structure to explain why a protected invariant may require re-verification.

34. Adversarial Cases

Case	Required outcome
Same name, different address	Not automatically SAME.
Same address, changed provider ID + replacement evidence	REPLACED.
Two plausible matches	UNCERTAIN.
Trusted resource absent in candidate	DELETED.
Candidate-only resource	CREATED.
Security-rule ordering changes only	UNCHANGED after canonicalization.
Unsupported dynamic dependency	UNKNOWN/UNSUPPORTED.
Missing edge	Not proof of no dependency.

35. Performance

The MVP is expected to operate on small controlled fixtures. Correctness, determinism, and explainability take priority over micro-optimization.

- Index resources for near-linear matching.
- Use sets/maps for edge deduplication.
- Canonicalize before expensive comparisons.
- Avoid live AWS calls during graph construction unless explicitly required by a supported relationship.
- Persist reports for downstream reuse.

36. Testing

36.1 Identity Tests

- Exact-address unchanged.
- Exact-address modified.
- Create/delete.
- Replacement.
- Provider-ID change.
- Ambiguous correspondence.
- Contradictory evidence.
- Equivalent normalization.
- Unsupported resource.

36.2 Dependency Tests

- Reference extraction.
- VPC/subnet.
- Subnet/route association.
- EC2/SG.
- RDS/SG.
- Rule ownership.
- IGW route relationship.
- Invariant scope edges.
- Unknown/unsupported relationship.

36.3 Determinism Tests

- Repeated identical input produces identical semantics/hash.
- Resource order does not alter result.
- Rule order does not alter semantic graph.

37. Acceptance Criteria

- Identity classifications are deterministic within the supported model.
- Exact supported address matches work correctly.
- Replacement is distinguished when evidence establishes recreation.

- Ambiguity is explicit UNCERTAIN.
- Created/deleted resources are correct.
- Supported references become typed graph edges.
- Supported infrastructure relationships become typed edges.
- Invariant scope is represented.
- Unknown/unsupported relationships are preserved.
- Reports contain explainable evidence and algorithm versions.

38. Non-Goals

- Complete AWS dependency graph.
- Runtime packet-level proof.
- Identity for unsupported resource types.
- Probabilistic identity based solely on LLM judgment.
- Impact closure/invalidation.
- Verification decisions.
- Promotion decisions.
- Automatic reconciliation of arbitrary provider behavior.

39. Final Identity/Dependency Directive

DAIL shall establish identity and dependency from explicit, reproducible evidence. Resource correspondence must be conservative, graph relationships must be typed and provenance-backed, and uncertainty must survive downstream rather than being erased by convenience heuristics.

The Identity Engine answers 'what changed?' The Dependency Engine answers 'what is connected to what, within the supported model?' Neither engine answers 'is the repair safe?' That remains the responsibility of Impact, Verification, and Promotion.

Appendix A — Output Summary

Output	Required contents
IdentityReport	Correspondence, identity status, change type, evidence, rationale, algorithm version.
DependencyGraph	Stable nodes/edges, edge types, provenance, uncertainty, graph version/hash.
IdentityEvidence	Facts supporting correspondence.
DependencyEvidence	Facts/rules supporting each edge.

Appendix B — Next Dependency

Document 08 — Impact & Invalidation Specification consumes the IdentityReport and DependencyGraph to determine which protected invariants require re-verification after a candidate transition.